REQUIREMENT QUALITY COPILOT

Complete Feature Description & Hackathon Planning Guide

Problem statement: PSE-01 — Requirement Quality Copilot
Suggested AI platform: Qualcomm Cloud AI Playground
Document purpose: Detailed descriptions of the proposed product features, example behavior, and implementation priorities.

Product vision

Transform rough product requirements into clear, consistent, reviewable, and verification-ready engineering specifications. The application should identify risks, explain the evidence, ask for missing decisions, suggest revisions, and keep engineers in control of approval.

Core workflow

Submit requirements → analyze quality → inspect evidence-backed issues → resolve missing decisions → review a rewrite → approve → generate a verification plan.

Design principles

- Evidence first: every issue should reference the relevant source wording and explain why it matters.
- Do not guess: missing thresholds, conditions, or stakeholder decisions should become clarification questions or explicit TBD fields.
- Human approval: AI suggestions remain drafts until an authorized reviewer accepts them.
- Explainable scoring: scores come from a documented rubric, not an unexplained model-generated number.
- Traceability: preserve original wording, requirement IDs, revisions, and decisions.

1. Requirement Input & Document Parser

Purpose: Accept requirements individually or as a document and convert them into stable, reviewable records.

- Support pasted text and individual requirement entry.
- Support CSV and TXT imports first; PDF parsing can be added if time and tooling allow.
- Split content into individual requirement statements and assign IDs such as REQ-001.
- Preserve the original wording and, where possible, source section or row.
- Allow users to add, edit, remove, and reorder requirements.
- Accept optional project context, glossary, operating conditions, and known constraints.
- Classify statements as functional, performance, security, constraint, or background information where feasible.

Example: “The device should start quickly. The device must store logs.” becomes REQ-001 and REQ-002, while retaining the original text.

Value: reliable IDs and preserved source text make issue evidence, conflict links, revisions, and exports traceable.

2. Requirement Quality Score

Purpose: Provide a transparent quality assessment for each requirement and for the overall document.

Dimension	Example weight	What it checks
Clarity	25%	Is the meaning sufficiently unambiguous?
Verifiability	25%	Can an objective verification approach be defined?
Completeness	20%	Are required parameters and conditions present?
Atomicity	15%	Does it express one obligation?
Consistency	15%	Does it fit with related requirements?

- Display a score out of 100 with a breakdown by dimension.
- Explain why points were deducted and link the explanation to specific evidence.
- Show document-level averages, number of requirements assessed, and open blockers.
- Recalculate after revision using the same rubric.
- Show unassessed dimensions separately rather than silently treating them as passing.

The weights are proposed design choices, not an official standard. A high score does not automatically mean the requirement is correct, approved, or technically feasible.

3. Requirement X-Ray — Issue Detection

Purpose: Highlight the exact text that makes a requirement risky and explain what action is needed.

Ambiguity: Flag subjective phrases such as “fast,” “robust,” “easy,” and “user-friendly.”

Unverifiability: Identify claims such as “excellent performance” that lack objective pass/fail criteria.

Incompleteness: Detect missing actors, triggers, conditions, boundaries, metrics, units, or expected outcomes.

Non-atomic statements: Flag requirements that combine independent obligations and may need to be split.

Unresolved placeholders: Find TBD/TBC fields, empty parameters, or references to undefined terms.

Each issue card should contain: issue type, severity, exact evidence quote, explanation, recommended action, clarification question where relevant, and status (open, resolved, or accepted with justification).

Creative extension: Make the highlighted phrase clickable so the reviewer can jump to the relevant part of the source document.

4. Duplicate Detection Engine

Purpose: Find repeated requirements even when their wording differs.

- Detect exact duplicates using conservative normalized text comparison.
- Find likely near-duplicates using lexical similarity or embeddings if available.
- Use semantic comparison to check whether statements express the same behavior under the same conditions.
- Group related requirements and show their IDs, shared intent, and differences.
- Recommend whether to merge, retain, or clarify; never merge automatically.

Example: “Send telemetry to the cloud every five seconds” and “Upload telemetry at five-second intervals” may be duplicates if scope and conditions match.

Creative extension: Duplicate Cluster Map showing related statements in one group. Similar-looking requirements may apply to different modes or product variants, so human confirmation is needed.

5. Conflict Radar

Purpose: Find requirements that may not be satisfiable together and show evidence from both statements.

- Compare actors, actions, scope, operating conditions, metrics, and units.
- Flag opposing behaviors or incompatible constraints when they apply to the same context.
- Show both requirement IDs, their exact evidence, the overlap in conditions, and the reason for concern.
- Offer a clarification question and possible resolution options.
- Record the selected resolution and re-analyze after the document changes.

Example: REQ-004 says the device shall transmit cloud telemetry during normal operation; REQ-009 says it shall make no external network transmissions during normal operation. Flag a potential conflict and ask whether telemetry is permitted in a separate connected mode.

> **Important safeguard:** Different values are not automatically contradictory. For example, two compatible upper bounds can both be satisfied. The detector must reason about scope and logical compatibility.

6. Smart Requirement Rewriter

Purpose: Propose clearer wording without silently changing the intended behavior.

- Improve clarity and precision.
- Replace vague terms with measurable criteria only when approved values or context are available.
- Split compound requirements when appropriate.
- Preserve actors, actions, conditions, constraints, and units.
- Display original and proposed wording side by side.
- Allow the user to edit, accept, or reject the suggestion.

Example: “The application should load quickly” becomes “The application shall display the dashboard within [TBD: maximum response time] under [TBD: operating conditions].”

Rewrite modes: Clarity mode, Testability mode, and Compare mode. Unspecified thresholds must remain TBD or trigger a clarification question.

7. Clarification Cards

Purpose: Turn missing information into explicit stakeholder decisions instead of fabricated AI assumptions.

- Generate focused questions for missing thresholds, triggers, operating conditions, scope, or definitions.
- Allow the user to enter a value, select a confirmed value from project context, or leave it unresolved.
- Record the answer and its source.
- Generate a revised requirement using confirmed answers.

- Re-run quality checks after clarification.
- Keep the requirement in draft status until approved.

Example: For “respond quickly,” ask for the maximum response time, the event that starts measurement, and the conditions under which the target applies.

Creative extension — Decision Memory: Maintain a list of confirmed project decisions and reuse them only when their scope is applicable and the source is visible.

8. Intent-Locked Rewrite & Change Validation

Purpose: Detect when a rewrite changes meaning, even if the new sentence sounds more polished.

- Compare actors, actions, objects, numerical values, units, conditions, exceptions, negation, and obligation strength.
- Show preserved details, changed details, and unresolved details.
- Flag material changes for review.
- Require explicit approval before accepting a rewrite that changes obligations or constraints.
- Retain original and approved versions for auditability.

Example: “The system shall not transmit diagnostic data while offline” must not be rewritten as “The system shall transmit diagnostic data while offline.” A changed negation is a major semantic difference.

> AI comparison can help, but no check guarantees perfect preservation. Human approval remains essential.

9. Verification Blueprint Generator

Purpose: Turn an approved requirement into a draft verification plan.

- Suggest a verification method: test, analysis, inspection, or demonstration.
- Generate preconditions, setup, procedure, expected results, and acceptance criteria.
- Identify required equipment or software where relevant.
- Generate positive, negative, and boundary test cases when appropriate.
- Link each test to its source requirement.
- Flag missing test-plan decisions and requirements without a verification method.
- Track verification planning separately from actual execution and verified status.

Example: For an approved 200 ms acknowledgement requirement, propose timestamped input/acknowledgement capture and a test procedure. The threshold must come from approved input; the test plan does not itself prove that the device passes.

10. Interpretation Duel

Purpose: Demonstrate how one ambiguous sentence can have multiple plausible meanings.

Example: “Retain recent diagnostic logs” might mean logs from the previous 24 hours or the latest 1,000 entries. Show both as illustrative interpretations, then ask whether retention should be based on time, count, storage capacity, or a combination.

- Display two or more plausible interpretations.
- Explain which missing phrase causes the ambiguity.
- Label interpretations as examples, not inferred stakeholder intent.
- Let the engineer select or define the intended meaning.
- Feed the answer into Clarification Cards and the rewrite workflow.

Value: A memorable visual demo of why vague requirements cause disagreement.

11. Change Impact Preview

Purpose: Show what other requirements or tests may need review when a requirement changes.

- Track known links between requirements, components, policies, and verification cases.

- Compare old and new versions.
- Suggest potentially affected items.
- Separate confirmed links from AI-inferred relationships.
- Allow engineers to confirm or dismiss suggested impacts.

Example: Changing log retention from 24 hours to seven days may affect storage capacity, deletion behavior, export requirements, retention policy, and related tests.

Creative extension: An Impact Map showing the changed requirement and linked items. This is an advanced feature because useful impact analysis needs relationship data.

12. Requirement Lifecycle Tracker

Purpose: Make it clear what stage each requirement has reached.

- New — submitted but not analyzed.
- Needs clarification — essential information is missing.
- Under review — issues need resolution.
- Ready for approval — no known blockers remain, but human approval is pending.
- Approved — an authorized reviewer accepted the requirement.
- Verification planned — a verification plan has been reviewed.
- Verified — evidence confirms that the approved criteria were met.

Support filters by status, severity, category, and owner. Record reviewer decisions and revisions. Do not equate approval with verification.

Creative extension: Project Readiness Dashboard showing reviewed/total requirements, open conflicts, pending decisions, approved requirements, and verification status.

13. Supporting Features

Executive Summary: Summarize overall document quality, major risks, duplicate groups, potential conflicts, and unresolved decisions.

Issue Prioritization & Filters: Rank issues by severity, impact, and confidence; filter by category, status, ID, or owner.

Requirement Traceability: Link requirements to sources, decisions, related requirements, and verification cases.

Version History: Keep original, proposed, and approved versions; compare revisions and record who approved changes.

Report Export: Export requirements, scores, findings, decisions, and verification plans to CSV or JSON; PDF reports are optional.

Domain-Specific Review: Support approved terminology, glossaries, and constraints for domains such as IoT, automotive, or embedded systems.

14. Technical Capabilities Behind the Features

Capability	Role
Qualcomm AI inference	Natural-language analysis, explanations, questions, and proposed rewrites.
Rule-based validation	Exact duplicates, IDs, formatting, placeholders, and selected numerical checks.
Semantic similarity	Candidate detection for paraphrased duplicates and related requirements.
Structured output validation	Validate response fields, issue categories, and requirement IDs.
Evidence validation	Check that quoted evidence appears in the original source.
Intent-change detection	Flag changed numbers, units, negation, conditions, and obligation words.
Scoring engine	Apply a consistent documented rubric outside the model.

Capability	Role
Persistence and evaluation	Store versions and decisions; test issue detection, false positives, and rewrite faithfulness.

Confirm the exact models, endpoints, quotas, and structured-output capabilities available in the team's Qualcomm Cloud AI Playground account before finalizing implementation.

15. Recommended Build Priority

Priority	Features	Reason
P0 — Core MVP	Input/parser; quality score; X-Ray issue detection; duplicate detection; conflict detection; rewriting.	Directly addresses the problem statement.
P1 — Differentiators	Clarification Cards; Intent-Locked Rewrite; Verification Blueprint.	Makes the tool safer and more useful than a basic chatbot.
P2 — If time permits	Interpretation Duel; Lifecycle Tracker; export; executive summary.	Improves the demo and usability after the core works.
P3 — Future scope	Change Impact Preview; traceability graph; advanced version history; domain-specific intelligence.	Requires more data, integration, and testing.

Final recommendation

Build a reliable end-to-end workflow first. Prioritize evidence-backed issue detection, clarification questions, intent-preserving rewrites, and verification planning. Treat scores as transparent review indicators, conflicts as findings requiring evidence, and all AI rewrites as drafts until a human approves them.

Planning note: Feature descriptions, scoring weights, examples, and priorities in this guide are proposed product ideas, not official Infosys or Qualcomm requirements.